УНИВЕРСИТЕТ ИТМО

Дисциплина: Backend

ОТЧЁТ

по лабораторной работе №1

«Реализация REST API на основе boilerplate»

Выполнила: Софья Кисель

Группа: БР 1.1

Санкт-Петербург, 2026

1. Цель и постановка задачи

Цель лабораторной работы — реализовать работоспособный REST API на основе boilerplate для предметной области бронирования столиков в ресторанах. В работе используются модели предметной области, обработчики HTTP-запросов и бизнес-логика, необходимая для регистрации пользователей, работы с ресторанами и управления бронированиями. В качестве технологической основы используются Node.js, TypeScript, Express и TypeORM. Для хранения данных применяется PostgreSQL, а для аутентификации пользователей — JWT. Работу API необходимо было проверить на реальном запуске приложения и продемонстрировать основные сценарии взаимодействия с сервером.

2. Структура и архитектура приложения

Приложение разделено на несколько логических частей. В config/ находятся настройки приложения и подключения к базе данных. Каталог models/ содержит TypeORM-сущности. В controllers/ реализованы REST-обработчики. DTO отвечают за входные данные, middleware — за проверку JWT, а common/ и utils/ содержат общие классы, перечисления и функции работы с паролями. Такое разделение позволяет отделить описание данных от HTTP-логики и вспомогательных функций. PostgreSQL запускается в Docker-контейнере через Docker Compose. При старте сервер подключается к базе данных, а TypeORM синхронизирует структуру таблиц с описанием сущностей.

3. Пользователи и аутентификация

Для пользователей реализованы регистрация, авторизация и работа с профилем. Регистрация выполняется через POST /auth/register, вход — через POST /auth/login. После успешной авторизации сервер выдаёт JWT-токен, который используется для доступа к защищённым маршрутам. Для текущего пользователя доступны GET /users/me, PATCH /users/me и GET /users/me/reservations. Middleware получает токен из заголовка Authorization и проверяет его подпись с использованием секретного ключа приложения. Пароли не хранятся в открытом виде. Перед сохранением пользователя пароль хэшируется с помощью bcryptjs через TypeORM-подписчик UserSubscriber. Поле password_hash исключено из ответов API. В приложении предусмотрены четыре роли: Admin, User, Owner и Manager, которые используются для разграничения операций.

4. Работа с ресторанами

Основной объект предметной области — ресторан. API позволяет получать список ресторанов и данные конкретного ресторана, а авторизованным пользователям с соответствующими правами — создавать и изменять записи. Для GET /restaurants реализована фильтрация по городу, ценовой категории и типу кухни. Для построения такого запроса используется TypeORM QueryBuilder: базовый запрос выбирает верифицированные рестораны, после чего к нему добавляются дополнительные условия в зависимости от переданных параметров. Для проверки приложения были использованы тестовые данные: несколько ресторанов с разными городами, кухнями, ценовыми категориями и статусами модерации.

5. Бронирование столиков

Для бронирований реализованы создание, просмотр, изменение и отмена. Основные маршруты — POST /reservations, GET /reservations/:id, PATCH /reservations/:id и DELETE /reservations/:id. Операции пользователя с его бронированиями защищены JWT-аутентификацией. При создании сначала проверяется существование выбранного столика, затем количество гостей сравнивается с его вместимостью. После этого выполняется проверка временного конфликта: учитываются существующие бронирования со статусами Pending и Confirmed в заданном временном диапазоне. Это предотвращает повторное бронирование одного столика на пересекающееся время. При изменении и

отмене также учитывается ограничение по времени до начала бронирования. Отмена не удаляет запись физически, а переводит её в статус Cancelled, сохраняя историю операции.

6. Дополнительные модули и тестовые данные

Помимо основных сущностей, в проекте представлены кухни, столики, меню и позиции меню, отзывы, фотографии, владельцы и сотрудники ресторанов. Для соответствующих объектов предусмотрены модели и контроллеры. Для первоначального заполнения базы используется src/seed.ts. Скрипт создаёт тестовые роли, пользователей, кухни, рестораны, столики, бронирования и отзывы. Его можно запускать повторно без намеренного создания дубликатов, что удобно при локальной разработке и проверке API.

7. Запуск и тестирование API

Для локального запуска PostgreSQL был поднят через Docker Compose. Зависимости проекта установлены через npm. Команда npm run build успешно собирает TypeScript-код, а npm start запускает скомпилированное приложение. Сервер работает на порту 8000, документация Swagger доступна по адресу /api/docs/. Основные пользовательские сценарии были проверены через Swagger UI. В ходе проверки была выполнена регистрация тестового пользователя, авторизация и получение профиля, просмотр ресторанов и столиков, а также полный цикл работы с бронированием: создание, просмотр, изменение и отмена.

Результаты проверки

Метод / маршрут	Результат	Проверяемый сценарий
POST /auth/register	201	регистрация пользователя
GET /users/me	200	профиль авторизованного пользователя
GET /cuisines	200	список кухонь
GET /restaurants	200	список ресторанов
GET /restaurants/1	200	данные ресторана
GET /restaurants/1/tables	200	столики ресторана
POST /reservations	201	создание бронирования
GET /users/me/reservations	200	бронирования пользователя
PATCH /reservations/4	200	изменение бронирования
DELETE /reservations/4	200	отмена бронирования

В рамках тестирования было создано бронирование столика ресторана. После создания оно было получено через API, затем количество гостей было изменено с помощью PATCH-запроса. После этого выполнена отмена бронирования; запись сохранилась в базе данных со статусом Cancelled.

8. Вывод

В результате лабораторной работы реализован и проверен REST API для системы бронирования столиков в ресторанах. Приложение использует TypeScript, Express, TypeORM и PostgreSQL, поддерживает JWT-аутентификацию и ролевую модель, а также содержит бизнес-логику проверки вместимости столиков и временных конфликтов бронирований. Основные маршруты были проверены через Swagger UI. Отдельно был протестирован полный пользовательский сценарий: регистрация и авторизация, получение профиля, просмотр ресторанов и столиков, создание бронирования, его изменение и отмена.